

1 Il Sistema Operativo: concetti fondamentali e architetture

1.1 Il concetto di Sistema Operativo e i suoi Obiettivi

Un sistema operativo (OS) è fondamentalmente un insieme di programmi che si interpone tra le applicazioni dell'utente e il ferro della macchina (l'hardware). Il suo ruolo è gestire risorse logiche e fisiche cercando di mantenere in equilibrio due obiettivi contrastanti: l'astrazione (semplificare la vita all'utente) e l'efficienza (sfruttare l'hardware al massimo). C'è sempre un trade-off: un sistema estremamente astratto come Windows risulta meno efficiente, mentre sistemi come Unix offrono meno astrazione ma performance nettamente superiori.

1.2 L'evoluzione storica: la caccia all'efficienza

Tutta la storia degli OS è una corsa per non lasciare mai la CPU inattiva.

1.2.1 Prima generazione (1946-1955):

Zero OS. C'erano enormi calcolatori a valvole, l'operatore era anche il programmatore e il codice veniva inserito fisicamente interruttore per interruttore. L'efficienza era minima.

1.2.2 Seconda generazione (1955-1965):

Arrivano i transistor e si passa al batching: i programmi simili venivano raggruppati in lotti (batch) per evitare di dover ricaricare continuamente gli stessi compilatori. Per fare questo serve un "monitor residente" in memoria (il vero antenato dell'OS). Nasce però un problema: la CPU elabora i dati a una velocità disarmante rispetto alle lentissime periferiche di I/O, rimanendo spesso inattiva. Si introduce quindi il DMA (Direct Memory Access) e il meccanismo asincrono degli Interrupt, che permettono alla periferica di avvisare la CPU solo a lavoro finito, eliminando l'inefficiente polling (l'interrogazione continua della periferica da parte della CPU). Iniziano ad essere usati il buffering e lo spooling per accumulare i job direttamente su disco.

1.2.3 Terza generazione (1965-1980):

È qui che nascono i concetti moderni. Per sfruttare appieno la macchina, si introduce la multiprogrammazione: la CPU inizia a sovrapporre le fasi di elaborazione di un job con le fasi di I/O di un altro job. L'evoluzione successiva è il time sharing (o multitasking), in cui il tempo della CPU viene tagliato a fettine e distribuito a rotazione tra i vari programmi, dando l'illusione del parallelismo. Essendo che ora più programmi coesistono, nasce il problema della protezione. Vengono introdotti limiti di memoria, timer per non far monopolizzare la CPU a un singolo processo e, soprattutto, l'architettura dual mode: si dividono le operazioni in Modo USER (privo di privilegi) e Modo SUPERVISOR/KERNEL (accesso totale all'hardware).

1.2.4 Quarta generazione (1980-Oggi):

Si diversificano le architetture, nascono i PC, i sistemi di rete, quelli distribuiti, real-time ed embedded.

1.3 Le Componenti dell'OS

Il testo scompone l'OS in vari “gestori”:

1.3.1 Gestione dei Processi:

Il kernel non gestisce i programmi (che sono file statici), ma i processi, ovvero le istanze in esecuzione. Il processo ha codice, dati, un suo heap (per l'allocazione dinamica) e uno stack.

1.3.2 Gestione della Memoria e dell'I/O:

L'OS decide chi caricare in RAM e alloca lo spazio su disco. Nasconde la complessità fisica dell'I/O usando buffer e device driver specifici.

1.3.3 Gestione dei File e Protezione:

Il File System astrae la memoria di massa in file e directory logiche, mentre un sistema di protezione regola chi ha l'autorizzazione per accedere a queste risorse.

1.3.4 Le System Call:

Sono il ponte cruciale del sistema. Mentre l'utente usa la shell (che è un programma di sistema, ma non fa parte del kernel), i programmi chiedono “favori” al kernel tramite le System Call. Per invocare una System Call (ad esempio passando parametri via registri, tramite stack o puntando a una tabella in memoria), il processore genera un interrupt software (TRAP) che fa saltare l'esecuzione in modalità Kernel a uno specifico gestore, per poi tornare in modalità Utente a operazione finita.

1.4 Struttura Architeturale del Sistema Operativo

Non tutti gli OS sono costruiti allo stesso modo. Il capitolo elenca vari approcci storici e moderni:

1.4.1 Sistemi Monolitici:

Nessuna divisione. Tutto il sistema operativo gira in blocco unico. Estremamente veloce ed efficiente (poiché non ci sono sovraccarichi per passare da un modulo all'altro), ma un vero inferno di complessità se c'è da fare manutenzione.

1.4.2 Strutture Semplici:

Come l'MS-DOS originale, dove i programmi potevano scavalcare i driver e toccare direttamente l'hardware (non essendoci il dual-mode).

1.4.3 Sistemi a Livelli:

Suddivisione a strati concentrici, dal livello più basso (hardware) al più alto (interfaccia utente). Un livello i può comunicare solo con i livelli adiacenti. Grande modularità e facilità di debug, ma efficienza disastrosa per via dei troppi salti necessari per eseguire un'operazione.

1.4.4 Micro-kernel e Virtual Machine:

Si porta al minimo il nucleo centrale (il micro-kernel gestisce solo il salvataggio dei processi e la comunicazione base), spostando file system e driver nello spazio utente per renderli più stabili. Le Virtual Machine spingono l'astrazione al limite: il sistema simula l'hardware nudo, garantendo protezione totale e l'esecuzione di più OS sulla stessa macchina host, al prezzo di inevitabili cali di performance.

1.4.5 Sistemi Client-Server:

Tutto è visto come un processo. Se voglio leggere un file, il mio processo client chiede il servizio a un processo server (che gestisce il file system), e il kernel fa solo da "postino" instradando i messaggi. Ottimo per i sistemi distribuiti su rete.

2 Processi

Il Capitolo 2 ("Processi") definisce l'entità fondamentale del sistema operativo. Un processo non è un programma (che è un file statico su disco), ma la sua istanza in esecuzione, la quale evolve sequenzialmente elaborando dati e richiedendo risorse fisiche e logiche.

2.1 L'Immagine in Memoria e il PCB

Per poter essere eseguito, il programma viene caricato creando una struttura chiamata *immagine in memoria*, suddivisa in quattro segmenti specifici:

Codice (Text) La parte statica e immutabile che contiene le istruzioni in linguaggio macchina.

Dati Contiene tutte le variabili globali utilizzate.

Stack (Pila) Memoria temporanea usata per gestire le chiamate a funzione, i passaggi di parametri, gli indirizzi di ritorno e le variabili locali automatiche. Cresce dall'alto verso il basso.

Heap Area di memoria allocata dinamicamente a runtime (tramite chiamate come `malloc` e `free`). Cresce dal basso verso l'alto, andando incontro allo Stack.

Ogni processo è rappresentato all'interno del kernel dal **PCB (Process Control Block)** o TCB, un blocco dati che racchiude: lo stato attuale, il Program Counter (indirizzo della successiva istruzione), il contenuto dei registri della CPU, i limiti di memoria assegnati, le informazioni per lo scheduling (priorità e puntatori alle code), i dati di accounting (quanto tempo CPU ha consumato) e lo stato dell'I/O (es. la lista dei file aperti).

2.2 Gli Stati e le Transizioni del Processo

Un processo attraversa ciclicamente cinque stati:

- **Nuovo:** Il processo viene creato, il sistema costruisce l'immagine in memoria e il PCB, ammettendolo nel sistema.
- **Pronto (Ready):** Il processo è in coda e attende solo che gli venga assegnato un core della CPU.

- **In Esecuzione (Running):** Le istruzioni vengono elaborate dalla CPU. Questa transizione dallo stato “Pronto” è chiamata *dispatch*.
- **In Attesa (Waiting):** Il processo si blocca spontaneamente per attendere un evento (es. operazione di I/O). Viene rimosso dalla CPU e, al completamento dell’evento, tornerà nello stato “Pronto”.
- **Terminato (Finito):** Il compito è concluso e il processo viene chiuso.

Un processo passa da “In esecuzione” a “Pronto” anche tramite una transizione forzata detta *time-out*, quando esaurisce il tempo a sua disposizione sulla CPU (time-sharing).

2.3 Scheduling e Cambio di Contesto

L’obiettivo dei sistemi è mantenere sempre la CPU occupata (multiprogrammazione) e commutare rapidamente tra i processi per garantire interattività (time-sharing). I processi si dividono in:

- **I/O bound:** Molte brevi sequenze di operazioni sulla CPU alternate a lunghe attese per l’I/O.
- **CPU bound:** Poche ma lunghissime sequenze di elaborazione matematica sulla CPU.

I processi pronti vengono inseriti in una *Ready Queue* (una lista concatenata di PCB), mentre quelli in attesa di eventi finiscono nelle *Wait Queue* dei rispettivi dispositivi.

Se la memoria scarseggia, il sistema può usare lo **Swapping**: rimuove temporaneamente un processo dalla RAM spostandolo sul disco rigido (swapped out), riducendo il grado di multiprogrammazione, per poi ricaricarlo successivamente in memoria (swapped in) ripristinandone lo stato.

Il passaggio della CPU da un processo all’altro è il **Cambio di Contesto (Context Switch)**, gestito dal modulo *Dispatcher*. Il sistema salva i registri del processo uscente nel suo PCB, passa in modalità kernel, e carica i registri dal PCB del processo entrante. Il tempo impiegato per questa operazione (latenza di dispatch) è puro sovraccarico (overhead).

2.4 Creazione, Terminazione e Comunicazione

I processi formano un albero gerarchico, originato dal processo radice **systemd** (PID 1). Ogni processo è identificato dal suo PID.

2.4.1 Creazione

Avviene tramite la chiamata di sistema **fork()**. Il sistema genera un clone esatto del padre. La memoria viene copiata, non condivisa, quindi le modifiche future dell’uno non intaccheranno l’altro. La **fork()** restituisce 0 al processo figlio e il PID del nuovo figlio al processo genitore. Se fallisce, restituisce -1, segnalando errori come **EAGAIN** (risorse insufficienti) o **ENOMEM** (memoria esaurita per le strutture del kernel).

Successivamente, il figlio usa spesso una funzione della famiglia **exec()** per sovrascrivere il proprio spazio di indirizzamento con un nuovo programma caricato dal disco.

2.4.2 Terminazione

Il processo si chiude invocando `exit()` o venendo forzato dall'esterno. Le risorse fisiche vengono deallocate. Lo stato di uscita rimane però nel kernel finché il padre non lo raccoglie tramite la chiamata `wait()`.

- Se il padre non invoca la `wait()`, il figlio terminato diviene un processo **Zombie** (mantiene il PCB occupato).
- Se il padre muore prima del figlio, il figlio diviene **Orfano** e viene adottato dal processo `systemd/init`, che eseguirà periodicamente la `wait()` al posto del padre originario per ripulire il sistema.

2.4.3 Comunicazione tra processi (IPC)

I processi possono essere isolati o cooperanti. Si coopera per tre motivi: condivisione di informazioni, velocizzazione del calcolo (parallelizzando i task) o modularità del sistema.

La comunicazione (IPC – Inter Process Communication) avviene in due modi:

Memoria Condivisa Più veloce. Il kernel interviene solo per allocare la regione di RAM, poi i processi leggono e scrivono direttamente in quello spazio.

Scambio di Messaggi Più lento ma facile da implementare. Ideale per piccole quantità di dati. Il kernel fa da tramite passandosi i messaggi.

2.5 L'esecuzione del Kernel

Il sistema operativo stesso è un programma che deve essere eseguito. Vi sono tre architetture per gestirlo:

Kernel separato Esegue in uno spazio totalmente isolato e sempre in modalità kernel. Tipico dei sistemi più antichi.

Kernel in processi utente Il codice dell'OS è mappato nello spazio di indirizzamento condiviso di ogni processo. L'immagine in memoria del processo si allarga e include uno Spazio di indirizzamento condiviso e un Kernel Stack (usato durante le chiamate di sistema). In questo modo, invocare una System Call richiede solo un rapido cambio di modalità (Mode Switch) e non un oneroso Cambio di Contesto.

Kernel come processo Le funzioni dell'OS sono gestite come processi individuali in modalità protetta, separati dai processi utente. Questo modello, pur perdendo il vantaggio del semplice Mode Switch, sfrutta al meglio le moderne architetture multiprocessore, permettendo al processo utente di girare su una CPU mentre il processo dell'OS gira parallelamente su un'altra.

3 Thread e concorrenza

Il **Capitolo 3** (“Thread e concorrenza”) sposta il focus dal processo inteso come blocco monolitico a una visione più frammentata e dinamica dell'esecuzione. Se il processo è il “contenitore” che possiede le risorse, il vero motore che esegue le istruzioni sulla CPU è il **Thread** (o filo di esecuzione).

3.1 Anatomia di un Thread e Vantaggi della Concorrenza

Un processo tradizionale (definito “pesante”) è costituito da un singolo thread. Tuttavia, i sistemi e le applicazioni moderne (inclusi i kernel stessi, come si nota dal processo `kthreadd` in Linux) sono strutturati per essere **multithread**, ospitando più percorsi di controllo simultanei.

Questa architettura separa nettamente il possesso delle risorse dalla capacità di eseguire il codice. All’interno di un processo multithread:

- **Cosa è privato:** Ogni thread possiede un proprio identificatore (ID), un proprio Program Counter, un proprio set di registri della CPU e un proprio Stack (per le variabili locali e le chiamate a funzione). Nello specifico, ogni thread è rappresentato da un proprio **TCB (Thread Control Block)**, in contrapposizione al PCB globale del processo.
- **Cosa è condiviso:** Tutti i thread appartenenti allo stesso processo condividono la sezione del codice, la sezione dei dati (variabili globali) e le altre risorse di sistema, come i file aperti e i segnali.

La programmazione multithread garantisce quattro grandi categorie di vantaggi:

1. **Tempo di risposta (Responsiveness):** In un’applicazione interattiva, se un thread si blocca per un’operazione lunga o pesante, gli altri thread possono continuare a girare, mantenendo l’interfaccia reattiva per l’utente.
2. **Condivisione delle risorse:** I thread condividono nativamente la memoria del processo a cui appartengono. A differenza dei processi distinti, non hanno bisogno di complesse chiamate di sistema per la memoria condivisa o lo scambio di messaggi (IPC).
3. **Economia:** Allocare memoria e risorse per un intero nuovo processo è molto oneroso. Creare thread e gestirne il cambio di contesto (Context Switch) è nettamente più economico in termini di tempi e cicli macchina.
4. **Scalabilità:** In un’architettura multicore, un processo a singolo thread può girare su un solo core alla volta. I processi multithread, invece, possono distribuire i propri thread su più core fisici, eseguendoli in vero parallelismo.

3.2 Implementazioni: Thread Utente vs Thread Kernel

Una questione fondamentale del multithreading è: *se un singolo thread si blocca (ad esempio per attendere un dato dal disco), l’intero processo si blocca con lui?* La risposta dipende dal livello in cui i thread sono implementati.

Thread a livello utente (Pure user-level): La gestione dei thread è demandata a un’apposita libreria che risiede nello *User Space*, senza alcun supporto da parte del sistema operativo. Il kernel ignora del tutto l’esistenza di questi thread e vede il processo come una singola entità monolitica. Il difetto critico è che se un thread effettua una chiamata bloccante, il kernel blocca l’intero processo, paralizzando di fatto anche tutti gli altri thread “fratelli”.

Thread a livello kernel (Pure kernel-level): I thread sono gestiti e schedulati direttamente dal sistema operativo. Poiché il kernel è pienamente consapevole della

pluralità dei flussi di esecuzione, se un thread si blocca in attesa, lo scheduler può tranquillamente mandare in esecuzione sulla CPU un altro thread dello stesso processo. Quasi tutti i sistemi operativi moderni utilizzano questa implementazione.

3.2.1 Modelli combinati

Per unire i benefici di entrambi i mondi, esistono anche dei modelli combinati. In questi modelli, una libreria utente gestisce un gran numero di thread a livello utente, i quali vengono “mappati” e distribuiti su un numero inferiore (o uguale) di thread a livello kernel. Questo permette di avere la flessibilità e la leggerezza dei thread utente, appoggiandosi alla robustezza e al parallelismo dei thread kernel.

4 Scheduling della CPU

4.1 I Concetti Fondamentali e gli Scheduler

Il fine ultimo della multiprogrammazione è massimizzare l’uso della CPU, facendo in modo che un processo sia in esecuzione in ogni istante.

L’esistenza di un processo non è un flusso continuo, ma un’alternanza tra **sequenze di elaborazione (CPU burst)** e **sequenze di attesa per le periferiche (I/O burst)**. Esistono processi *I/O bound* (molti burst di CPU brevissimi) e *CPU bound* (pochi burst di CPU lunghissimi).

Il sistema gestisce il traffico avvalendosi di tre tipologie di Scheduler:

- **A breve termine (o della CPU):** Molto veloce, pesca costantemente dalla *ready queue* (la coda dei processi pronti) per assegnare il processore a un nuovo task.
- **A lungo termine:** Seleziona quali programmi ammettere nel sistema e caricare in RAM. Oggi è in gran parte desueto nei sistemi moderni.
- **A medio termine:** Entra in scena quando la RAM è intasata e attua lo *swapping*: “prende” alcuni processi, li parcheggia sul disco rigido (diminuendo il grado di multiprogrammazione) e li ricarica in memoria in un secondo momento.

Una distinzione architetturale vitale per lo scheduling è tra:

4.1.1 Senza prelazione (cooperativo):

Nel primo caso, il processo tiene la CPU finché non finisce o non richiede un evento di I/O.

4.1.2 Con prelazione (preemptive):

Nel secondo caso (usato in tutti i sistemi moderni), il sistema operativo può “strappare” la CPU al processo in esecuzione. Questo garantisce interattività, ma introduce complessità progettuali estreme nel kernel per evitare corruzioni di dati condivisi (*race condition*).

L’entità che materialmente ferma un processo, ripristina i registri del successivo e lo fa ripartire è il **Dispatcher**. Il tempo “sprecato” per compiere questa transizione è la *latenza di dispatch*.

4.2 Criteri di Valutazione

Come decidiamo se un algoritmo è buono? Dobbiamo bilanciare diverse metriche:

- *Da massimizzare:* l'**Utilizzo della CPU** e il **Throughput** (produttività, ossia quanti processi vengono terminati nell'unità di tempo).
- *Da minimizzare:* il **Tempo di completamento** (Turnaround time, l'intervallo tra invio e fine), il **Tempo di attesa** (il tempo totale trascorso fermo nella *ready queue*) e il **Tempo di risposta** (quanto ci mette il sistema a iniziare a rispondere).

4.3 Gli Algoritmi di Scheduling

Il capitolo analizza poi le tattiche con cui il sistema operativo organizza la *ready queue*.

4.3.1 FCFS (First-Come, First-Served):

Gestione FIFO basilare. Chi arriva primo prende la CPU. È senza prelazione ed è afflitto dall'*effetto convoglio*: processi brevissimi rimangono in coda ad aspettare che un singolo processo lunghissimo finisca i suoi calcoli, abbassando drasticamente le prestazioni globali.

4.3.2 SJF (Shortest-Job-First):

Assegna la CPU al processo che ha il successivo *CPU burst* più breve. Matematicamente è l'algoritmo ottimale per avere il minor tempo di attesa medio, ma c'è un problema di fondo: non si può conoscere in anticipo il futuro. La soluzione pratica è fare una "predizione" calcolando una **media esponenziale** delle durate passate (dando maggior peso storico agli eventi più recenti). Ne esiste anche una versione con prelazione (nota come *Shortest-Remaining-Time-First*).

4.3.3 Round Robin (RR):

Concepito appositamente per il *time-sharing*. A ogni processo viene data una fettina di tempo (il **Time slice** o *Quanto di tempo*, es. 10-100 ms). Se il processo non ha finito quando scade il "quanto", interviene la prelazione e il processo finisce in fondo alla coda circolare. Le sue prestazioni dipendono dalla dimensione del quanto: troppo grande lo fa regredire a un FCFS, troppo piccolo lo affoga nella latenza dei continui cambi di contesto.

4.3.4 Scheduling con priorità:

Si serve per primo il processo con la priorità maggiore. Causa un problema grave chiamato **Starvation** (attesa indefinita): in un sistema carico, un processo a bassa priorità potrebbe non ricevere mai la CPU. Il rimedio è l'**Aging** (invecchiamento): la priorità di un processo in coda viene progressivamente aumentata col passare del tempo. Una sua variante dinamica è l'algoritmo **HRRN**, che calcola le priorità includendo direttamente nella formula il tempo speso ad aspettare.

4.3.5 Code multilivello:

Si spezza la *ready queue* in code diverse a priorità fissa (es. una per i task di sistema, una per quelli interattivi, una per i *batch*). Ognuna usa l'algoritmo più adatto a sé, ma le code alte hanno sempre precedenza su quelle basse.

4.3.6 Code multilivello con retroazione:

I processi non sono vincolati a una coda a vita, ma possono “spostarsi”. Se un processo consuma troppo tempo CPU (comportandosi da CPU bound), viene declassato a una coda di priorità inferiore. Se attende troppo, viene promosso per evitare *starvation*.

4.3.7 Fair Share:

Abbandona la visione del singolo processo. Se un'applicazione usa più processi, le risorse non vengono calcolate a livello di processo, ma a livello di “gruppi” di applicazioni, garantendo che le risorse vengano divise equamente tra le *app* in esecuzione.

4.4 Valutazione degli Algoritmi

Infine, come sceglie un ingegnere il giusto algoritmo? Ci sono tre vie:

4.4.1 Modello deterministico:

Si prende un carico di lavoro predefinito (una tabella di processi con burst e arrivi noti) e si calcola su carta il tempo medio di attesa usando diagrammi di Gantt.

4.4.2 Modello a reti di code:

Basato su modelli matematico-probabilistici degli arrivi. Sfrutta regole come la *formula di Little* ($n = \lambda \times W$) per calcolare la stabilità e la lunghezza delle code nel sistema.

4.4.3 Simulazione:

Si programma un modello software dell'OS. Per avere una simulazione altamente precisa, si scartano i numeri casuali e si registrano le esecuzioni di un sistema reale creando un nastro di tracciamento (**trace tape**), in modo da alimentare il simulatore con dati 100% veritieri.

5 Sincronizzazione dei Processi

5.1 L'Origine del Problema: Le Race Condition

Quando due processi cooperanti condividono dei dati (come una variabile globale in memoria condivisa) e tentano di modificarli simultaneamente, si va incontro a un disastro chiamato Race Condition (condizione di corsa).

Il testo usa l'esempio di un produttore e un consumatore che aggiornano una variabile contatore. Se il produttore fa `contatore++` e il consumatore fa `contatore--` nello stesso esatto momento, a livello di linguaggio macchina le istruzioni si intrecciano in modo

casuale. Il risultato finale sarà un numero errato e incoerente, perché abbiamo permesso a entrambi di manipolare lo stesso dato in modo concorrente senza alcuna “regola del traffico”.

5.2 Il Problema della Sezione Critica

Per evitare il caos, si definisce Sezione Critica quel segmento di codice in cui un processo accede a dati condivisi (modifica tabelle, scrive su file, ecc.). La regola d’oro è: se un processo è nella sua sezione critica, nessun altro può entrare nella propria.

Una soluzione valida a questo problema deve garantire tre requisiti inviolabili:

Mutua esclusione: Mai più di un processo contemporaneamente nella sezione critica.

Progresso: Se la sezione critica è libera e qualcuno vuole entrarci, la decisione su chi entra non può essere rimandata all’infinito.

Attesa limitata: Deve esserci un limite al numero di volte che gli altri processi possono sorpassare un processo che sta aspettando di entrare, evitando che quest’ultimo muoia di inedia (starvation).

In base a come gestiscono questo problema al proprio interno, i kernel si dividono in senza prelazione (immuni alle race condition ma lentissimi) e con prelazione (veloci e adatti al real-time, ma difficilissimi da progettare perché richiedono protezioni continue sui dati del kernel).

5.3 La Soluzione di Peterson

Storicamente, si provò a risolvere il problema via software. La soluzione di Peterson vale per due processi (P_0 e P_1) e usa due variabili condivise: un array `boolean flag[2]` (per indicare l’intenzione di entrare) e un `int turn` (per indicare di chi è il turno).

La logica è elegante: il processo dichiara di voler entrare (`flag[i] = true`), ma per cortesia cede il turno all’avversario (`turn = j`). Poi aspetta in un ciclo `while` finché l’altro non ha finito.

Questa soluzione garantisce matematicamente i tre requisiti, ma ha un enorme difetto pratico: sulle architetture moderne, i compilatori e le CPU ottimizzano e “riordinano” l’esecuzione delle istruzioni, rompendo la logica dell’algoritmo.

5.4 Il Supporto Hardware

Visto che il software da solo non basta, l’hardware deve fornire aiuti concreti.

Barriere di memoria: Sono istruzioni speciali che bloccano il riordino delle istruzioni da parte della CPU, forzando la propagazione visibile delle modifiche alla memoria a tutti gli altri processori.

Istruzioni Atomiche: I processori offrono istruzioni come `test_and_set()` e `compare_and_swap()`. Sono operazioni hardware eseguite in modo “indivisibile” (atomico): leggono e modificano un valore in un colpo solo, senza che nessun interrupt possa interromperle a metà.

Variabili Atomiche: Sfruttando le istruzioni sopra citate, si creano tipi di dato basilari (es. interi) le cui operazioni (come l'incremento) sono sempre sicure e atomiche.

5.5 Lock Mutex

Sfruttando le istruzioni hardware, i progettisti dei sistemi operativi creano strumenti per i programmatori. Il più semplice è il Lock Mutex (da Mutual Exclusion).

Funziona con due funzioni: `acquire()` per prendere il lucchetto (entrando nella sezione critica) e `release()` per rilasciarlo uscendo.

Il problema del mutex classico è che l'`acquire()` implementa un'attesa attiva (busy waiting): se il lucchetto è occupato, il processo continua a ciclare all'infinito consumando cicli CPU a vuoto. Questo tipo specifico di lucchetto è chiamato Spinlock. Ha il difetto di sprecare CPU, ma il pregio di non causare un cambio di contesto (che, come sai, fa perdere tempo), rendendolo perfetto sui sistemi multiprocessore se l'attesa dura pochissimo.

5.6 I Semafori

I semafori sono strumenti molto più potenti e robusti del semplice lucchetto. Un semaforo è una variabile intera `S` accessibile solo tramite due operazioni atomiche:

`wait(S)`: Se `S` è ≤ 0 , aspetta. Altrimenti, decrementa `S` di 1.

`signal(S)`: Incrementa `S` di 1.

Si dividono in semafori binari (funzionano esattamente come i mutex, con valori 0 o 1) e semafori contatore (usati per gestire una risorsa presente in multipli esemplari, es. 5 stampanti).

Come si elimina l'attesa attiva? Invece di far girare la CPU a vuoto nel ciclo `wait`, si ridefinisce il semaforo dotandolo di una coda di attesa interna.

Se un processo fa una `wait` su un semaforo occupato, il sistema lo "addormenta" (mettendolo in stato di waiting) e lo inserisce nella coda del semaforo, liberando la CPU per un altro processo. Quando un altro processo fa una `signal`, il sistema fa una `wakeup()`, sveglia il primo processo in coda e lo riporta nella ready queue.

Con questa implementazione, il valore intero del semaforo può scendere sotto zero: se il valore è negativo, il suo valore assoluto indica esattamente quanti processi stanno dormendo in coda.

5.7 Un problema classico: I Cinque Filosofi

Il capitolo si chiude con un celebre problema di sincronizzazione. Ci sono `N` filosofi e `N` bacchette. Per mangiare, un filosofo deve prendere sia la bacchetta a sinistra che quella a destra.

Soluzione intuitiva (Errata): Si usa un semaforo per ogni bacchetta. Il filosofo fa una `wait` sulla bacchetta sinistra e poi sulla destra. Il problema è il Deadlock: se tutti i filosofi prendono la bacchetta sinistra nello stesso istante, nessuno potrà mai prendere la destra. Il sistema si paralizza.

Soluzione corretta: Si usa un vettore che traccia lo stato esatto di ogni filosofo (THINKING, HUNGRY, EATING). Si usa un mutex generale per proteggere i cambiamenti di stato. Un filosofo può passare allo stato EATING (e quindi farsi dare il via libera dal semaforo) solo e unicamente se i suoi due vicini (destro e sinistro) non stanno già mangiando. In questo modo si evitano stalli fatali.

6 Processi, programmi e memoria

La prima distinzione fondamentale è tra programma e processo. Un programma è semplicemente un file binario su disco contenente istruzioni, mentre un processo è l'istanza dinamica di quel programma in esecuzione.

Per il kernel, un processo è formato dalla memoria *user-space* (dove risiede il codice), dalle variabili utilizzate e da una serie di strutture dati interne (come la tabella delle pagine o dei file aperti) usate per gestirne lo stato e le risorse.

6.1 Lo schema in memoria (Linux x86-32)

Quando il processo viene caricato in RAM, la sua memoria viene divisa in sezioni ben precise:

- **Program code (text):** Un segmento a sola lettura che contiene le istruzioni in linguaggio macchina.
- **Initialized data (data):** Contiene le variabili globali e statiche che sono state esplicitamente inizializzate nel codice.
- **Uninitialized data (bss):** Contiene le variabili globali e statiche dichiarate ma non inizializzate.
- **Heap:** L'area dedicata all'allocazione dinamica della memoria (es. tramite `malloc`), che cresce dal basso verso l'alto.
- **Stack:** L'area usata per le chiamate a funzione, il passaggio dei parametri e le variabili locali, che cresce dall'alto verso il basso incontrandosi idealmente con l'heap.

Il testo riprende anche il classico tranello dei puntatori: inizializzare una stringa con `char *string = "ciao"` salva il testo nel segmento *program code* a sola lettura; tentare di modificarla causa un errore di segmentazione. Al contrario, usare un array come `char string[] = "ciao"` alloca i dati direttamente sullo stack, rendendoli liberamente modificabili.

6.2 La Tabella dei Descrittori di File

Per gestire l'input/output, il sistema operativo crea per ogni processo una *file descriptor table*, dove vengono salvati tutti i file aperti a run-time.

Di default, ogni nuovo processo nasce con tre file descriptor pre-assegnati:

- 0: standard input (*stdin*)
- 1: standard output (*stdout*)

- 2: standard error (*stderr*)

I nuovi file aperti dal programmatore prenderanno sempre il numero intero libero più basso disponibile (partendo dal 3 in poi).

6.3 System Calls (Chiamate di Sistema)

Le system call sono funzioni speciali che il programma invoca per chiedere favori al kernel (come creare processi o aprire file). Senza di esse è impossibile interagire con l'hardware o le risorse di sistema. Sono ampiamente documentate nella sezione 2 del manuale Unix (*man 2*).

6.3.1 Come vengono eseguite dietro le quinte?

L'applicazione utente chiama una funzione “wrapper” (un contenitore) presente nella libreria standard del C (*glibc*).

1. Questa funzione copia gli argomenti nei registri della CPU e inserisce un numero identificativo (che rappresenta la specifica chiamata di sistema) nel registro *%eax*.
2. Viene generato un interrupt che fa commutare la CPU dalla modalità utente alla modalità kernel.
3. Il kernel salva lo stato, legge il numero della chiamata, cerca in una tabella di puntatori la routine di servizio associata e la esegue.
4. Terminato il lavoro, il kernel ripristina i registri, posiziona il risultato dell'operazione nello stack e restituisce il controllo al wrapper, tornando in modalità utente.

6.3.2 La Gestione degli Errori

In caso di fallimento, la chiamata di sistema restituisce solitamente -1 (o NULL per i puntatori). Contestualmente, la funzione wrapper imposta in automatico una variabile globale chiamata *errno* (dichiarata in *<errno.h>*) con un codice numerico specifico che identifica la natura dell'errore.

Per tradurre questo numero in testo comprensibile si usano funzioni come *perror()* (che stampa il messaggio su standard error) o *strerror(errno)* (che restituisce la stringa descrittiva dell'errore).

Per analizzare a fondo il comportamento di un programma senza leggerne il codice sorgente, si può usare da terminale il comando *strace* (*strace <comando>*), che intercetta e stampa a video in sequenza tutte le chiamate di sistema effettuate in tempo reale dal processo.

6.4 Tipi di dato del Kernel e Manuale

Per garantire la massima portabilità del codice su architetture hardware e versioni di Linux diverse, il kernel utilizza tipi di dati personalizzati definiti nell'header *<sys/types.h>*. Sono facili da riconoscere perché terminano tutti con *_t* (ad esempio *pid_t* per gli ID dei processi, *size_t* per le dimensioni, *off_t* per la posizione nei file, ecc.).

Infine, il capitolo ribadisce l'importanza delle pagine di manuale (`man <comando>`), suddivise in sezioni numerate:

- Sezione 1: Comandi utente.
- Sezione 2: System call.
- Sezione 3: Funzioni della libreria C.

È anche possibile cercare un termine specifico all'interno dei manuali usando il comando `man -k <stringa>`.

7 Il VFS e la filosofia “Everything is a file”

In Linux domina il principio per cui “tutto è un file”. Che si tratti di un testo, di un dispositivo hardware (come una tastiera) o di una connessione di rete, il kernel espone un'interfaccia uniforme. Questo miracolo di astrazione è reso possibile dal **Virtual File System (VFS)**: uno strato software intermedio che riceve le chiamate di sistema generiche e le “traduce” nelle operazioni specifiche richieste dal file system fisico sottostante (es. ext4, FAT32).

7.1 Operazioni Base sui File

Il ciclo di vita di un file all'interno di un programma si basa su una sequenza standard di chiamate di sistema.

7.1.1 Apertura e Creazione (open):

La funzione `open(pathname, flags, mode)` serve sia per aprire file esistenti sia per crearne di nuovi.

- I `flags` stabiliscono la modalità di apertura (es. `O_RDONLY` per la sola lettura, `O_WRONLY` per la scrittura, `O_CREAT` per crearlo se non esiste, `O_TRUNC` per svuotarlo all'apertura).
- Il `mode` definisce i permessi (es. `S_IRUSR`, `S_IWUSR`). Tuttavia, i permessi effettivi applicati al nuovo file dipendono dall'`umask` del sistema, che agisce come un filtro sottraendo i permessi di default da quelli richiesti dal programmatore.
- Restituisce il file descriptor o `-1` in caso di errore.

7.1.2 Lettura (read):

La funzione `read(fd, buf, count)` tenta di leggere `count` byte, salvandoli nel buffer puntato da `buf`. Sposta automaticamente in avanti il cursore di lettura. Un dettaglio vitale: non è un errore se `read` restituisce meno byte di quelli richiesti (succede spesso con le connessioni di rete); se restituisce 0, significa che si è raggiunta la fine del file (EOF).

7.1.3 Scrittura (`write`):

La funzione `write(fd, buf, count)` scrive i dati dal buffer al file. Lo standard POSIX impone che i dati appena scritti siano immediatamente disponibili per una successiva lettura.

7.1.4 Spostamento del cursore (`lseek`):

Ogni file aperto ha un cursore interno. La funzione `lseek(fd, offset, whence)` sposta questo cursore senza leggere o scrivere nulla. L'argomento `whence` decide il punto di riferimento:

- `SEEK_SET`: dall'inizio del file
- `SEEK_CUR`: dalla posizione attuale
- `SEEK_END`: dalla fine del file

7.1.5 Chiusura (`close`):

Libera il file descriptor e le risorse associate. Anche se il kernel chiude in automatico i file quando un processo termina, è buona pratica chiuderli esplicitamente per forzare la scrittura fisica dei dati bufferizzati su disco.

7.1.6 Rimozione (`unlink`):

Elimina il “nome” del file dalla directory. I blocchi fisici di memoria (l'inode) verranno deallocati dal disco solo quando non ci saranno più hard link che puntano a esso e nessun processo lo starà ancora tenendo aperto.

7.2 Caratteristiche e Permessi dei File

I metadati di un file non sono scritti nel file stesso, ma nel suo “inode”. Per ispezionarli, la libreria C offre una famiglia di funzioni dedicata.

7.2.1 Lettura dei metadati (`stat`, `lstat`, `fstat`):

Popolano una struttura dati di tipo `struct stat`. Questa struttura è una miniera di informazioni. Contiene:

- `st_ino`: il numero dell'inode.
- `st_size`: la grandezza logica del file in byte.
- `st_blocks`: lo spazio fisico effettivamente occupato sul disco, espresso in blocchi da 512 byte.
- `st_mode`: un campo fondamentale che racchiude sia il tipo di file (regolare, directory, link simbolico) sia i permessi. Si usano apposite macro (come `S_ISREG(st_mode)`) per verificare rapidamente di che tipo di file si tratta.

7.2.2 Permessi:

Sono divisi sui canonici tre livelli: Proprietario (User), Gruppo (Group) e Altri (Other), con permessi di Lettura (r), Scrittura (w) ed Esecuzione (x).

7.2.3 Verifica d'accesso (access):

Controlla se il processo ha i requisiti per leggere (R_OK), scrivere (W_OK) o eseguire (X_OK) un file. A differenza delle normali operazioni, **access** basa il suo controllo sull'UID reale dell'utente che ha lanciato il programma, non sull'UID effettivo.

7.2.4 Modifica dei permessi (chmod, fchmod):

Permettono di sovrascrivere i permessi applicando una nuova maschera bit a bit (es. S_IRUSR | S_IWUSR).

7.3 Gestione delle Directory

Le directory sono file speciali che contengono liste di nomi associati ai rispettivi inode. Poiché la loro struttura deve rimanere integra, il kernel impedisce ai programmatori di scriverci dentro direttamente con la normale funzione **write**. Bisogna usare un set di chiamate dedicate.

7.3.1 Creazione e Rimozione (mkdir, rmdir):

- **mkdir(dirname, mode)** crea una directory vuota (contenente solo i riferimenti standard `.` e `..`).
- Per eliminarla si usa **rmdir(pathname)**, ma l'operazione ha successo solo se la directory è rigorosamente vuota e non è in uso da alcun processo.

7.3.2 Lettura del contenuto (Gli Stream):

POSIX utilizza il concetto di "directory stream".

1. Si apre lo stream con **opendir(dirpath)**, che restituisce un puntatore speciale di tipo `DIR *`.
2. Si legge il contenuto con **readdir(dirp)**. Ogni chiamata restituisce una struttura `struct dirent` contenente i dati di una singola voce della directory (come il nome `d_name` e il tipo `d_type`).
3. Per leggere tutta la cartella, si inserisce **readdir** in un ciclo **while** che si ferma solo quando la funzione restituisce `NULL` (che indica la fine delle voci).
4. Terminata la lettura, lo stream va chiuso con **closedir(dirp)**.

8 Attributi e Anatomia di un Processo

Un processo non è solo codice in esecuzione, ma porta con sé un bagaglio di identificatori e contesti fondamentali per il kernel.

8.0.1 PID e PPID

Ogni processo possiede un ID univoco (PID) assegnato in ordine progressivo fino a un massimo di 32768. Conserva anche la memoria del processo che lo ha generato (PPID, Parent Process ID). Il PID numero 1 è sempre riservato al processo capostipite del sistema (`init` o `systemd`). Si leggono tramite `getpid()` e `getppid()`.

8.0.2 UID/GID Reale vs Effettivo

Linux esegue un doppio controllo sull'identità.

- L'**UID reale** (`getuid()`) rappresenta l'utente fisico che ha lanciato il programma.
- L'**UID effettivo** (`geteuid()`) è invece il “pass pass” usato dal kernel per controllare se hai i permessi per aprire un file o compiere un'azione. Se un file ha il bit SUID impostato, quando lo esegui, l'UID effettivo diventa temporaneamente quello del proprietario del file (spesso l'amministratore), permettendoti azioni privilegiate.

8.0.3 Le Variabili d'Ambiente

Ogni processo eredita una “lista d'ambiente”, cioè un array di stringhe nel formato `NOME=valore`, terminato da un puntatore NULL. Il codice C può accedervi tramite la variabile globale `extern char **environ` oppure passandola come terzo parametro del `main` (`char *env[]`). Per modificarle a runtime si usano le funzioni `getenv()`, `setenv()` e `unsetenv()`.

8.0.4 Directory di lavoro

Il kernel tiene traccia dell'inode della cartella da cui il processo sta operando. Si legge con `getcwd()` e si cambia con `chdir()` (passando un percorso) o `fchdir()` (passando un file descriptor di una directory già aperta).

9 I Descrittori di File e la Redirezione

La tabella dei file descriptor mappa i numeri interi alle risorse fisiche di I/O usate dal processo. I primi tre descrittori (0, 1 e 2 per `stdin`, `stdout` e `stderr`) vengono aperti di default.

9.0.1 La funzione `dup(oldfd)`

Crea un duplicato di un descrittore esistente. La regola aurea di Linux è che assegna sempre il primo numero intero libero disponibile. I due descrittori diventano interscambiabili, condividendo sia i lock sia il cursore di lettura/scrittura.

9.0.2 Il trucco della redirezione

Se chiudi lo standard output (`close(1)`) e poi apri subito un file normale, il kernel assegnerà a quel file il descrittore numero 1 (perché è il primo libero). Da quel momento, ogni tua normale `printf` finirà scritta dentro il file invece che sullo schermo.

10 Creazione, Terminazione e Monitoraggio

È qui che si gestisce l'albero genealogico dei processi.

10.0.1 Uscita Pulita vs Immediata

Un programma termina normalmente con un `return` dal `main` o chiamando `exit(status)`: questa funzione svuota i buffer I/O, chiude gli stream ed esegue, in ordine inverso (LIFO), tutte le eventuali funzioni di “pulizia” registrate in precedenza dal programmatore tramite la chiamata `atexit()`. Se invece usi `_exit(status)`, scavalchi tutto: il controllo passa direttamente al kernel, si chiude istantaneamente il processo e si rischia di perdere i dati rimasti nei buffer non salvati.

10.0.2 La `fork()`

Genera un figlio che è il clone del padre (stesso codice, dati e stack). La memoria viene copiata, non condivisa, quindi le modifiche alle variabili fatte da uno non si riflettono sull'altro. Restituisce 0 al figlio e il PID del figlio al padre. Se non c'è più RAM o si è raggiunto il limite massimo di processi, fallisce e restituisce -1.

10.0.3 Raccogliere l'eredità (`wait` e `waitpid`)

Un padre deve aspettare che il figlio termini per raccoglierne lo stato d'uscita (*status*), altrimenti il figlio morto rimarrà nel sistema come processo “Zombie”.

- La `wait(&status)` è bloccante: il padre si ferma e aspetta il primo figlio che muore.
- La `waitpid(pid, &status, options)` è molto più potente: permette di aspettare un figlio con uno specifico PID e, attivando l'opzione `WNOHANG`, permette di “chiedere” se il figlio è morto senza però bloccarsi ad aspettare.

10.0.4 Decodificare lo *status*

Lo *status* catturato non è un semplice intero, ma un pacchetto di bit. Per decifrarlo si usano delle macro apposite:

- `WIFEXITED` ti dice se il figlio si è chiuso normalmente, e in tal caso `WEXITSTATUS` ti restituisce il codice numerico (es. 0 o 1).
- `WIFSIGNALED` ti dice se il figlio è stato ucciso in modo anomalo (es. tramite un segnale del sistema operativo), e `WTERMSIG` ti rivela quale segnale lo ha ucciso.

11 La Famiglia `exec`

La `fork()` clona il processo, ma per fargli fare un lavoro completamente diverso si usa la famiglia di funzioni `exec`.

Quando chiami un `exec`, l'immagine in memoria del tuo programma viene letteralmente cancellata e sovrascritta caricando dal disco il nuovo file eseguibile. Il PID resta uguale, ma il programma cambia. Se la `exec` ha successo, non ritornerà mai al codice sorgente che l'ha invocata, proprio perché quel codice è stato cancellato dalla RAM.

Esistono 6 varianti di questa funzione, e il loro suffisso (le lettere finali del nome) è una vera e propria legenda per capire come passargli i parametri:

- l (list) vs v (vector):** Indicano come passi gli argomenti (es. “ls”, “-l”). Con la **l** (**exec1**) passi gli argomenti scritti uno per uno direttamente tra le parentesi tonde della funzione, terminandoli con un NULL. Con la **v** (**execv**) li inserisci preventivamente in un vettore/array e passi il vettore.
- p (path):** Se la funzione ha la **p** (**exec1p**, **execvp**), il sistema cercherà automaticamente il file da eseguire usando la variabile d’ambiente **PATH** del sistema operativo (esattamente come fa la shell). Senza la **p**, devi passargli il percorso assoluto o relativo perfetto (es. `/bin/ls`).
- e (environment):** Se è presente la **e** (**execle**, **execve**), la funzione ti permette di passare un array personalizzato di variabili d’ambiente per il nuovo programma. Se non c’è la **e**, il nuovo programma si terrà automaticamente le variabili d’ambiente che aveva il processo che lo ha invocato.

12 Sincronizzazione e Deadlock

Il capitolo 1 introduce concetti cruciali e un po’ ostici, come la gestione della sincronizzazione e dei deadlock in ambiente concorrente, riprendendo i semafori (spesso croce e delizia per ogni studente) e spiegando strumenti più strutturati come i monitor.

Vediamo di scomporre tutto in maniera chiara, discorsiva e dettagliata, senza perdere nessun concetto chiave.

12.1 Il problema alla base: I limiti dei Semafori

Come hai studiato nella prima parte di teoria, i semafori sono variabili speciali usate per regolare l’accesso a sezioni critiche di codice (usando le funzioni **wait()** per entrare e **signal()** per uscire). Il Capitolo 1 si apre subito mettendo in guardia su un fatto: **usare i semafori è difficile e rischioso**. Essendo comandi liberi, la correttezza del programma dipende interamente dalla disciplina del programmatore.

Basta un singolo errore in un solo processo per mandare in tilt tutto il sistema. Qualche esempio classico:

- **Inversione:** Se scrivi prima **signal()** e poi **wait()**, la protezione sparisce: più processi entreranno nella sezione critica contemporaneamente, causando il caos (race condition).
- **Doppia attesa:** Se per sbaglio metti due **wait()** di fila, il processo si bloccherà per sempre aspettando sé stesso.
- **Dimenticanza:** Se dimentichi di fare una **signal()**, blocchi fuori tutti gli altri processi a tempo indeterminato.

12.2 Il Costrutto Monitor: La soluzione elegante

Per ovviare alla potenziale “anarchia” dei semafori, i linguaggi di programmazione di alto livello hanno introdotto il **Monitor**, un Tipo di Dato Astratto (ADT). Puoi immaginare

il monitor come una “stanza sicura” che incapsula al suo interno variabili condivise e le funzioni per operare su di esse.

La regola d’oro del monitor è semplicissima: **il costruito stesso garantisce che solo un processo alla volta possa essere attivo al suo interno**. Il programmatore non deve più preoccuparsi di mettere a mano i lucchetti (i `wait` e i `signal`) per garantire la mutua esclusione, perché è il monitor che se ne occupa automaticamente. Tutto ciò che è fuori dal monitor non può vedere o toccare i dati che stanno all’interno.

12.2.1 Le Variabili Condition

Nonostante la mutua esclusione garantita, un processo all’interno del monitor potrebbe aver bisogno di aspettare che accada un evento prima di poter procedere (ad esempio, un consumatore che aspetta che il buffer si riempia). Per questo si usano le **variabili di tipo condition**.

Queste variabili non contengono numeri o valori (non puoi fare $x = 7$), ma servono solo per accodare i processi. Su di esse si usano due operazioni:

- **`wait()`**: Il processo che la invoca si sospende (si mette a dormire) su quella variabile, cedendo il “posto” all’interno del monitor in modo che qualcun altro possa entrarvi.
- **`signal()`**: Serve a svegliare un processo sospeso su quella variabile. Se non c’è nessuno sospeso, **l’operazione non fa assolutamente nulla** (a differenza della `signal` dei semafori classici, che invece incrementa sempre un contatore).

Un dettaglio sottile ma cruciale: se il processo P fa una **`signal`** per svegliare il processo Q, chi deve continuare a girare dentro il monitor? Esistono due compromessi: o P si ferma e aspetta che Q finisca (“segnalare e attendere”), oppure Q aspetta che P esca dal monitor (“segnalare e proseguire”).

Limiti dei Monitor: Sebbene siano comodissimi, i monitor hanno due problemi: pochi linguaggi li supportano nativamente (Java è uno di questi, tramite la keyword `synchronized` e i metodi `wait()` e `notify()`) e, soprattutto, **richiedono l’esistenza di una memoria condivisa**. Se i processi girano su macchine fisiche diverse collegate in rete (sincronizzazione non globale), il monitor non si può usare e si deve ricorrere allo scambio di messaggi.

12.3 I Deadlock: L’incubo dei sistemi operativi

Nella sincronizzazione, il nemico supremo è il **Deadlock (o stallo)**. *Cos’è?* È la situazione in cui un gruppo di processi si blocca in modo definitivo perché ogni processo sta aspettando un evento (il rilascio di una risorsa) che può essere causato solo da un altro processo di quello stesso gruppo bloccato. Immagina un ingorgo su un ponte: l’auto A non può andare avanti se l’auto B non si sposta, e l’auto B non può andare avanti se l’auto A non si sposta.

Affinché si crei fisicamente un deadlock, **devono verificarsi contemporaneamente 4 condizioni**:

1. **Mutua esclusione:** Le risorse in gioco non possono essere usate da più processi allo stesso tempo.

2. **Hold and Wait (Possesso e attesa):** Un processo tiene già in mano una risorsa e, intanto, ne aspetta un'altra che è occupata da qualcun altro.
3. **No preemption:** Nessuno (nemmeno il Sistema Operativo) può rubare la risorsa con la forza al processo che la detiene; deve essere lui a rilasciarla volontariamente.
4. **Attesa circolare:** Si crea una catena chiusa in cui P1 aspetta una risorsa di P2, P2 aspetta una risorsa di P3... e P3 aspetta una risorsa di P1.

Per studiare queste situazioni si usa uno strumento visuale molto comodo chiamato **Resource Allocation Graph (RAG)**. In questo grafo, i cerchi sono i processi e i quadrati sono le risorse. Una freccia dal processo al quadrato è una “richiesta”, una freccia dal quadrato al processo indica che la risorsa è “assegnata”. La regola è semplice: **se nel grafo non ci sono cicli, non c'è deadlock**. Se ci sono cicli ed esiste solo una copia per tipo di risorsa, allora il deadlock c'è di sicuro.

12.4 Le strategie contro il Deadlock

I Sistemi Operativi usano vari approcci per gestire questo problema, che vanno dal filosofico all'ingegneristico.

12.4.1 A. Algoritmo dello Struzzo (Non fare niente)

È l'approccio usato in realtà da quasi tutti i sistemi moderni (inclusi Windows e Linux): siccome gestire i deadlock costa troppa CPU e avvengono raramente, il sistema fa finta di niente. Se si verifica, toccherà all'utente uccidere il processo a mano.

12.4.2 B. Prevenzione Statica

Ingegnierizza il sistema in modo da **rendere impossibile per definizione almeno una delle 4 condizioni** viste prima.

- Per bloccare l'*Hold and Wait*: costringi il processo a chiedere tutte le risorse che gli serviranno all'inizio dell'esecuzione. Spreca molte risorse e causa inefficienza.
- Per bloccare il *No preemption*: se un processo chiede una risorsa occupata, il sistema gli strappa via con la forza tutte quelle che aveva già accumulato. Funziona solo per cose salvabili (come la CPU), ma non puoi farlo con una stampante a metà stampa.
- Per bloccare l'*Attesa Circolare*: è la via più usata. Si numerano tutte le risorse del computer. I processi sono obbligati a richiedere le risorse **solo in ordine numerico crescente**. Questo trucco matematico rende impossibile la formazione di un cerchio logico.

12.4.3 C. Prevenzione Dinamica (L'Algoritmo del Banchiere)

Invece di mettere regole rigide, il sistema analizza la situazione momento per momento prima di concedere una risorsa. Il sistema cerca sempre di rimanere in uno “**Stato Safe**” (**Stato Sicuro**): una situazione in cui esiste almeno una combinazione di passi che permette a tutti i processi di finire il loro lavoro, per quanto complessa possa essere. Se

non esiste una strada sicura, il sistema è in stato *Unsafe* (che non significa per forza deadlock, ma significa “pericolo di deadlock”).

Quando un processo chiede una risorsa, il sistema (come un **Banchiere** che deve decidere se concedere o meno un prestito ai suoi clienti in modo da non restare mai a secco di contanti) finge di dargliela e fa i calcoli. Il sistema usa quattro matrici per fare i conti:

1. Risorsa al momento *Disponibili*.
2. Richiesta *Massima* di ogni processo.
3. Risorsa già *Assegnate* a ogni processo.
4. *Necessità* residua di ogni processo (Massimo - Assegnate).

Il sistema cerca di capire se c'è un processo che può finire il suo lavoro con le sole risorse *Disponibili* attuali. Se lo trova, immagina che questo processo finisca e restituisca le risorse (andando ad aumentare i *Disponibili*), e poi ripete la verifica per un altro processo. Se riesce a trovare una sequenza che fa finire tutti i processi (Sequenza Safe), concede la risorsa. Se la simulazione sbatte contro un muro, nega la risorsa e mette il processo in attesa.

12.4.4 D. Rilevazione e Ripristino

Se non fai prevenzione o evitamento, prima o poi il deadlock capiterà. In questo caso, il sistema fa girare periodicamente un algoritmo di **Rilevazione** per cercare se si sono formati dei cicli nel grafo delle risorse occupate. Se scopre che c'è un deadlock, deve attuare un **Ripristino**, e qui le opzioni sono brutali e costose (perché il lavoro fatto finora viene buttato via):

- **Uccidere i processi:** O ammazza tutti i processi coinvolti (azzerando tutto) o ne uccide uno alla volta rifacendo il controllo per vedere se il deadlock sparisce (un'operazione pesantissima).
- **Prelazione (Rollback):** Ruba una risorsa a un processo e lo riporta indietro nel tempo a uno stato di salvataggio (checkpoint) precedente. Questo però rischia di generare *starvation* se rubi le risorse sempre allo stesso sfortunato processo.

13 Gestione della Memoria

13.1 Introduzione e Hardware di Base

La memoria centrale è un enorme array di byte a cui la CPU accede per prelevare istruzioni e dati. Dato che l'accesso alla RAM richiede molti cicli di clock (a differenza dei registri interni), si interpone una memoria cache veloce direttamente sul chip.

Per far convivere più processi è necessaria la protezione hardware: si usano un registro base (che contiene l'indirizzo fisico di partenza) e un registro limite (la grandezza del blocco). Ogni indirizzo generato dalla CPU in modalità utente viene confrontato con questi registri; in caso di violazione, il sistema lancia un'eccezione (trap) fatale.

13.2 Associazione degli Indirizzi (Binding)

Gli indirizzi dei programmi devono essere mappati in memoria fisica. Questo avviene in tre possibili fasi:

- **Tempo di compilazione:** Se si sa già dove il processo finirà, si genera codice assoluto.
- **Tempo di caricamento:** Il compilatore genera codice rilocabile, e l'associazione fisica viene fatta dal loader.
- **Tempo di esecuzione:** Permette di spostare i processi dinamicamente. È il metodo moderno e richiede la MMU (Memory-Management Unit).

La MMU divide definitivamente l'indirizzo logico (virtuale, generato dalla CPU) dall'indirizzo fisico (reale, visto dalla RAM). La MMU somma all'indirizzo logico il valore di un registro di rilocazione per ottenere l'indirizzo fisico esatto.

13.3 Linking e Caricamento Dinamico

Per risparmiare spazio, si utilizzano librerie condivise (DLL). Con il linking dinamico, il codice non ingloba l'intera libreria, ma solo uno stub (un riferimento). La libreria viene caricata in memoria solo quando chiamata (a runtime), permettendo a più processi di condividerne un'unica copia in RAM.

13.4 Allocazione Contigua della Memoria

In questo modello, un processo occupa un singolo blocco di memoria ininterrotto.

- **Partizioni fisse:** La memoria è divisa a priori in blocchi di dimensione fissa. È facile da gestire ma limita il grado di multiprogrammazione e causa frammentazione interna (spazio sprecato dentro la partizione se il processo è più piccolo).
- **Partizioni variabili:** Il sistema operativo tiene traccia dei “buchi” di memoria libera. Per assegnare i buchi si usano gli algoritmi First-fit (il primo abbastanza grande), Best-fit (il più piccolo in grado di contenerlo) o Worst-fit (il buco più grande). First-fit e Best-fit sono i migliori, ma questo approccio genera frammentazione esterna: i buchi si sparpagliano nella memoria, sprecando fino al 33% dello spazio (regola del 50%). La soluzione è la compattazione (unire i buchi spostando i processi), ma costa molta CPU ed è fattibile solo con rilocazione dinamica.
- **Buddy System:** Un compromesso. La memoria viene divisa iterativamente a metà (in blocchi di dimensione 2^k) finché non si trova il blocco minimo sufficiente. Vantaggio: la compattazione è velocissima perché i blocchi “gemelli” si riuniscono in un attimo. Svantaggio: torna la frammentazione interna, poiché le richieste vengono arrotondate a potenze di due.

13.5 Paginazione

La paginazione abbandona il concetto di allocazione contigua. La memoria fisica viene divisa in blocchi fissi chiamati frame, mentre la memoria logica è divisa in pagine della stessa esatta dimensione.

Un indirizzo logico è diviso in numero di pagina (p) e offset (d). Il numero p fa da indice in una Tabella delle Pagine, che restituisce il numero del frame fisico f . L'indirizzo fisico finale è semplicemente f unito a d .

Vantaggi: Azzera la frammentazione esterna e permette l'uso di spazi virtuali più grandi della memoria fisica.

Svantaggi: La frammentazione interna rimane (mediamente mezza pagina per processo).

13.5.1 Supporto Hardware (TLB)

Se la tabella delle pagine fosse salvata in RAM, ogni lettura richiederebbe due accessi alla memoria (uno per la tabella, uno per il dato). Per ovviare a questo, si usa la TLB (Translation Look-aside Buffer), una cache hardware associativa piccolissima e superveloce che memorizza le mappature pagina-frame più recenti.

Se la pagina è nella TLB (TLB hit), l'accesso è immediato. Se manca (TLB miss), bisogna leggere la tabella in RAM, penalizzando il tempo d'accesso effettivo (EAT). La TLB usa degli identificatori (ASID) per sapere a quale processo appartengono le traduzioni, garantendo protezione.

13.5.2 Pagine Condivise e Protezione

La tabella delle pagine usa bit di validità per impedire accessi illegali ed eventuali bit di protezione (Read/Write). Un vantaggio gigantesco è la condivisione: mappando la stessa pagina fisica su tabelle logiche diverse, decine di processi possono condividere una singola istanza di codice “rientrante” (come le librerie C), risparmiando enormi quantità di memoria.

13.6 Struttura della Tabella delle Pagine

All'aumentare dei bit dell'architettura, una singola tabella diventa enorme.

- **Gerarchica (a 2 livelli):** L'indirizzo virtuale viene spezzato in due indici (uno per la tabella esterna, uno per quella interna) più l'offset. Funziona bene a 32 bit, ma è catastrofica sui sistemi a 64 bit perché richiederebbe troppi livelli e troppi accessi sequenziali in memoria.
- **Tabella Hash:** Usata per spazi oltre i 32 bit. L'indirizzo passa in una funzione hash che punta a una lista concatenata contenente le mappature.
- **Tabella Invertita:** Ribalta il concetto: esiste una sola tabella globale con una riga per ogni frame fisico installato nel computer (contenente la coppia $\langle \text{PID}, \text{pagina} \rangle$). Risparmia una quantità enorme di RAM, ma rende le ricerche molto lente, costringendo l'uso congiunto di una tabella hash per velocizzarle.

13.7 Segmentazione

La Segmentazione suddivide la memoria in base alla logica del programmatore (un segmento per lo stack, uno per le subroutine, ecc.). I segmenti hanno lunghezza variabile. L'indirizzo è formato da $\langle \text{numero_segmento}, \text{offset} \rangle$ e la tabella dei segmenti conserva

l'indirizzo fisico di base e la lunghezza massima (limite). Facilita enormemente la condivisione e la protezione logica, ma riporta in vita il problema della frammentazione esterna a causa dell'allocazione dinamica di lunghezze variabili.

13.8 Segmentazione Paginata

Per unire il meglio dei due mondi nasce la Segmentazione Paginata. In questa architettura ibrida, gli elementi logici restano suddivisi in segmenti, ma ogni segmento è a sua volta suddiviso in pagine. La tabella dei segmenti non punta più a un indirizzo fisico contiguo in RAM, ma punta all'indirizzo base della Tabella delle Pagine specifica per quel segmento. Questo stratagemma elimina del tutto la frammentazione esterna, mantenendo intatta la visione logica strutturata del programmatore.

14 Gestione della Memoria Virtuale

14.1 Il Concetto di Memoria Virtuale

La memoria virtuale astrae la memoria centrale in un vettore enorme, separando la memoria logica percepita dall'utente da quella fisica reale. Questo permette di eseguire programmi che sono più grandi della memoria fisica disponibile, caricandone solo le parti necessarie. I vantaggi sono molteplici: un aumento dell'uso e della produttività della CPU (multiprogrammazione), minori operazioni di I/O al caricamento, e l'offerta di uno spazio degli indirizzi "sparso". Questi spazi sparsi lasciano ampi "buchi" di memoria tra heap e stack per permetterne la crescita e facilitano enormemente la condivisione di librerie di sistema (come la `libc` in sola lettura) e file tra più processi concorrenti.

14.2 Paginazione su Richiesta (Demand Paging)

Il cuore della memoria virtuale è la paginazione su richiesta: una pagina viene caricata dal disco alla RAM solo nel momento esatto in cui viene richiesta dall'esecuzione del programma.

Per implementarla si usa un bit di validità all'interno della tabella delle pagine: se è "valido" la pagina è in memoria, se è "non valido" la pagina si trova nella memoria secondaria (spazio di swap).

Se il processo tenta l'accesso a una pagina non valida, l'hardware lancia una trap al sistema operativo detta page fault. La gestione segue questa procedura:

1. Si controlla se il riferimento è legale; se lo è, si individua un frame libero in RAM.
2. Si programma un'operazione su disco per trasferire la pagina nel frame.
3. Si aggiorna la tabella delle pagine segnando la pagina come "valida".
4. Si riavvia l'esatta istruzione che era stata interrotta, in modo del tutto trasparente per il programma.

Le prestazioni del sistema dipendono dal Tempo d'accesso effettivo (EAT). Indicando con p la probabilità di page fault e con ma il tempo d'accesso standard alla memoria, la formula è:

$$EAT = (1 - p) \cdot ma + p \cdot \text{tempo di gestione del page fault}.$$

Poiché il tempo di gestione del disco è nell'ordine dei millisecondi, basta un tasso di page fault impercettibile per rallentare l'intero calcolatore di un fattore di 40.

14.3 Sostituzione delle Pagine e Algoritmi

In una condizione di sovrallocazione, la RAM si riempie e non ci sono più frame liberi per i nuovi page fault. Il sistema deve espellere una “pagina vittima”. Per non raddoppiare i tempi di I/O, l'hardware usa un bit di modifica (dirty bit): la pagina vittima viene scritta su disco solo se è stata alterata in memoria, altrimenti viene semplicemente scartata.

La scelta della vittima è dettata da specifici algoritmi testati su “successioni di riferimenti” per minimizzare il tasso di page fault:

14.3.1 FIFO (First-In First-Out)

Sostituisce la pagina presente in memoria da più tempo tramite una coda circolare. Il suo difetto critico è l'Anomalia di Belady: per alcune sequenze, aumentare la RAM (numero di frame) fa paradossalmente aumentare il numero di page fault.

14.3.2 Ottimale (OPT / MIN)

Sostituisce la pagina che non verrà usata per il periodo di tempo più lungo nel futuro. È l'algoritmo perfetto e non soffre di anomalie, ma è teorico e inapplicabile nella realtà perché il kernel non può prevedere il futuro.

14.3.3 LRU (Least Recently Used)

Sostituisce la pagina non usata da più tempo nel passato, approssimando l'OPT. È un “algoritmo a stack” (immune all'anomalia di Belady), ma è difficile da implementare perché richiede di aggiornare pesanti contatori hardware o stack di puntatori a ogni singolo accesso in memoria.

14.3.4 Approssimazioni LRU

Sfruttano un bit di riferimento impostato a 1 dall'hardware a ogni lettura/scrittura. L'algoritmo più famoso è quello con Seconda Chance (Clock): la memoria è vista come una lista circolare scansionata da un puntatore; se la pagina analizzata ha il bit a 1, questo viene azzerato (seconda chance) e si passa alla successiva, ma se è a 0 viene immediatamente sostituita.

14.4 Allocazione dei Frame

I frame liberi devono essere ripartiti tra i processi. Esiste un numero minimo di frame per processo dettato rigidamente dall'architettura del processore (es. per contenere l'istruzione e i suoi operandi indiretti). L'allocazione della rimanenza può essere Uniforme (ogni processo riceve la stessa frazione m/n di frame) o Proporzionale (i frame si spartiscono in base alla dimensione dello spazio virtuale richiesto da ciascun processo).

14.5 Thrashing (Degenerazione)

Se a un processo viene allocato un numero di frame inferiore a quelli che sta attivamente usando, incorrerà in una sequenza continua di page fault. Questa situazione si chiama thrashing: il processo perde più tempo a scambiare pagine con il disco che a eseguire istruzioni. Questo fa crollare l'utilizzo della CPU; lo scheduler reagisce erroneamente immettendo nuovi processi nel sistema, collassando del tutto le prestazioni.

Per fermare il thrashing si sfrutta il principio di Località (un programma esegue il codice spostandosi in moduli o insiemi di dati ben definiti):

- **Modello del Working-Set:** Si osserva una finestra fissa temporale degli ultimi Δ riferimenti. L'insieme delle pagine referenziate in quella finestra costituisce il "working set" del processo. Se la somma dei working set di tutti i processi supera i frame totali del sistema, si va in thrashing. Il kernel risolve la crisi sospendendo un processo e prelevando tutti i suoi frame.
- **Controllo della Frequenza (PFF):** Si misurano i page fault nel tempo. Se superano un limite massimo di soglia, si assegna d'urgenza un nuovo frame al processo; se scendono sotto un limite minimo, gli si sottraggono i frame in eccedenza per riassegnarli.

15 Gestione della Memoria di Massa

Il Capitolo 4 analizza l'architettura dei dispositivi di memoria di massa e le logiche di scheduling per ottimizzarne i tempi di lettura e scrittura.

15.1 Struttura dei Dispositivi di Memorizzazione

- **Nastri magnetici:** Supporti plastici ad accesso strettamente sequenziale, oggi relegati a funzioni di backup a causa della grandissima latenza di riposizionamento della testina (che può richiedere decine di secondi).
- **Dischi magnetici (HDD):** Composti da piatti rotanti (con velocità da 5400 a 15000 RPM) e testine di lettura mobili. La superficie è divisa in tracce circolari e settori (l'unità minima di trasferimento, dai 512 B ai 4 kB). Il tempo di accesso totale è dato dal seek time (il tempo di ricerca per spostare il braccio sul cilindro), dalla latenza di rotazione e dal tempo di trasferimento.
- **Dispositivi NVM (Non-Volatile Memory):** Memorie a stato solido (come gli SSD su base NAND) prive di parti meccaniche. Azzerano la latenza di rotazione e il seek time, collegandosi spesso direttamente al bus PCIe del sistema. Non possono sovrascrivere direttamente i dati senza prima eseguire una lunga operazione di cancellazione dell'intera cella e sono soggetti a deterioramento fisico (massimo 100.000 cicli per blocco).

15.2 Scheduling dei Dischi Rigidi

Per massimizzare la larghezza di banda e ridurre i tempi di ricerca meccanica, i driver di dispositivo intercettano e riordinano le richieste di I/O usando specifici algoritmi:

- **FCFS (First-Come First-Served):** Rispetta l'ordine di arrivo in coda. È equo ma altamente inefficiente, poiché rischia di far fare continui “salti” meccanici estremi al braccio del disco.
- **SSTF (Shortest Seek Time First):** Sceglie sempre la richiesta che comporta il minore spostamento possibile dalla posizione attuale della testina. Velocizza i trasferimenti ma rischia di creare starvation (inedia) per le richieste periferiche.
- **SCAN (Algoritmo dell'ascensore):** La testina viaggia da un'estremità all'altra del disco, esauendo tutte le richieste incontrate sul percorso prima di invertire la marcia al bordo.
- **C-SCAN (Circular SCAN):** Simile allo SCAN, ma una volta arrivata al bordo la testina scatta immediatamente all'inizio del disco senza servire richieste durante il ritorno, garantendo così tempi di attesa più uniformi.
- **C-LOOK:** Ottimizzazione del C-SCAN: il braccio inverte la rotta non appena non ci sono più richieste in quella direzione, evitando di spingersi inutilmente fino al bordo estremo del disco.
- **N-step SCAN e FSCAN:** Suddividono la coda delle richieste in sotto-code rigide per evitare che l'arrivo continuo di nuove richieste vicine “intrappoli” la testina nella stessa zona spaziale.
- **LIFO (Last-In First-Out):** Serve le ultime richieste arrivate per massimizzare la località dei dati, ma presenta un rischio di starvation catastrofico.

15.3 Gestione delle Unità e Aree di Memoria

- **Formattazione Fisica (Basso livello):** Eseguita in fabbrica, suddivide il disco vergine in settori inserendo un'intestazione e codici ECC per la correzione automatica degli errori.
- **Partizioni e Volumi:** Il disco logico viene spezzettato in partizioni che il sistema gestisce come unità a sé stanti, o raggruppate in Volumi.
- **Formattazione Logica:** È la creazione formale del file system vero e proprio, in cui il sistema registra le strutture dati iniziali come le directory vuote e la tabella dello spazio libero.
- **Blocchi Difettosi:** Sui dischi vecchi, il kernel marca i blocchi guasti per evitarli. Nei dischi avanzati, il controllore hardware esegue silenziosamente l'accantonamento dei settori (sector sparing): devia la lettura/scrittura da un settore rotto a un blocco di riserva nascosto, tenendo all'oscuro il sistema operativo.
- **Area di Avvicendamento (Swap):** Utilizzata per espandere virtualmente la RAM sul disco. Per arginare la lentezza cronica dell'I/O, le partizioni di swap sono spesso allocate in blocchi grezzi e saltano la struttura a file system per fornire il miglior throughput possibile al gestore della memoria virtuale.

16 Gestione dei File

16.1 Il Concetto di File e la Tabella dei File Aperti

Il file è l'unità logica minima di memorizzazione in memoria secondaria. Un file non possiede solo il suo contenuto, ma un pacchetto di attributi fondamentali: nome, identificatore unico (ID), tipo, locazione (puntatore al dispositivo), dimensione, protezione e metadati temporali (data di creazione, ultima modifica).

Per ottimizzare le operazioni (creazione, lettura, scrittura, ricerca/seek, troncamento), il sistema operativo utilizza la tabella dei file aperti per evitare di scorrere continuamente la directory a ogni singola richiesta. Quando un file è in uso, questa tabella mantiene in RAM informazioni vitali:

- **Puntatore al file:** Traccia l'offset logico corrente per la successiva operazione di lettura/scrittura; questo puntatore è separato e indipendente per ogni processo.
- **Contatore dei file aperti:** Registra quante chiamate `open()` sono attualmente attive sul file, permettendo al sistema di deallocare la voce in tabella solo quando il contatore scende a zero a seguito dell'ultima `close()`.
- **Posizione su disco e diritti d'accesso:** Mantiene l'indirizzo fisico in memoria e la modalità in cui il file è stato aperto, in modo da verificare i permessi a ogni operazione di I/O senza rileggere il disco.

16.2 Metodi di Accesso

Esistono due paradigmi principali per scorrere i dati di un file:

- **Accesso Sequenziale:** Eredità dei nastri magnetici. Le informazioni vengono lette o scritte rigorosamente in ordine tramite un cursore che avanza automaticamente (`read_next`, `write_next`).
- **Accesso Diretto:** Sfrutta l'architettura dei dischi magnetici. Il file è visto come una sequenza numerata di blocchi o record a lunghezza fissa, permettendo ai programmi di saltare in ordine casuale da un blocco logico all'altro senza alcuna restrizione sequenziale.

16.3 L'Evoluzione della Struttura delle Directory

La directory è la struttura che indicizza e organizza i file e deve supportare operazioni di ricerca, creazione, attraversamento e ridenominazione. Il suo design logico si è evoluto nel tempo:

16.3.1 A un livello

Un unico calderone per tutti i file. In sistemi multi-utente genera collisioni sui nomi, dato che l'identificativo deve essere globalmente univoco.

16.3.2 A due livelli

Introduce una Master File Directory (MFD) che smista la ricerca in singole User File Directory (UFD) dedicate ai singoli utenti. Risolve le collisioni di denominazione, ma isola gli utenti tra loro e introduce la necessità di usare i pathname (percorsi).

16.3.3 Ad albero

Generalizza il sistema permettendo sottocartelle illimitate a partire da una singola directory radice (root). Consente di gestire file tramite percorsi assoluti (dalla radice) e percorsi relativi (dalla directory corrente di lavoro del processo).

16.3.4 A grafo aciclico

Permette la condivisione di file o cartelle tramite i link (collegamenti) anziché duplicarne fisicamente il contenuto. Mantiene la semplicità degli algoritmi di ricerca perché previene la chiusura di loop infiniti, vietando la formazione di cicli strutturali.

16.3.5 A grafo generale

Ammette la creazione di cicli (autoriferimenti) all'interno dell'albero. Questo costringe il kernel a limitare arbitrariamente la profondità di ricerca per evitare loop infiniti e impedisce l'uso dei semplici "contatori di riferimenti" per le cancellazioni. Per deallocare lo spazio dei dischi in questa struttura è necessaria una complessa routine di garbage collection periodica che marchi e recuperi i blocchi orfani.

16.4 Protezione

La salvaguardia dei dati si divide in protezione fisica (gestita tramite copie di backup per prevenire danni hardware) e logica (prevenzione di accessi impropri). I sistemi moderni attuano un controllo granulare permettendo o negando tipi di accesso specifici: lettura, scrittura, esecuzione, aggiunta, cancellazione ed elencazione. Questo meccanismo è implementato principalmente tramite le Access Control List (ACL), strutture associate a ogni file che definiscono esattamente i privilegi concessi ai singoli utenti e gruppi. Quando un utente richiede un'operazione, il sistema la incrocia con la ACL prima di autorizzarla.

17 Segnali

Il Capitolo 8 analizza in dettaglio i Segnali, il meccanismo con cui il kernel notifica ai processi il verificarsi di eventi specifici (come errori o scadenze di timer) o con cui i processi comunicano in forma elementare tra loro.

17.1 Concetti Base e Semantiche

Gli eventi che generano un segnale spaziano da errori del programma (es. divisione per zero), alla terminazione di un processo figlio, fino a richieste esplicite dell'utente (come la pressione di combinazioni di tasti). Alla ricezione di un segnale, il processo interrompe il normale flusso di esecuzione per eseguire un'azione predefinita o una funzione personalizzata detta signal handler (intercettazione del segnale).

La gestione dei segnali si è evoluta in due modelli:

- **Semantica inaffidabile (sistemi Unix originari):** Il gestore non rimane attivo dopo l'esecuzione e il programmatore deve reinstallarlo manualmente. Questo espone il sistema a race condition: se arriva un secondo segnale prima della reinstallazione, viene eseguita l'azione di default, causando la potenziale perdita del segnale o la terminazione del processo. Inoltre, non c'è modo di bloccare i segnali temporaneamente per proteggere le esecuzioni critiche.
- **Semantica affidabile (sistemi Unix/Linux moderni):** Il gestore resta installato permanentemente. Un segnale generato viene consegnato (eseguendo l'azione) oppure rimane pendente qualora il processo decida di bloccarlo temporaneamente.

17.2 Classificazione dei Segnali

I segnali si dividono in sincroni (generati da un'azione specifica del programma e ricevuti in quel preciso momento, come un'eccezione matematica) e asincroni (imprevedibili, generati da eventi esterni come operazioni I/O). Anche se internamente il kernel usa identificatori numerici, i programmatori devono usare le macro standardizzate in `<signal.h>` per garantire la portabilità del codice:

- **Terminazione:** `SIGTERM` richiede una chiusura formale e pulita (può essere intercettato o bloccato). `SIGINT` e `SIGQUIT` sono generati da tastiera (rispettivamente con `CTRL-c` e `CTRL-\`); `SIGQUIT` si distingue perché genera automaticamente anche un core dump. L'eccezione assoluta è `SIGKILL`: impone la terminazione forzata brutale e non può mai essere intercettato, ignorato o bloccato.
- **Allarme:** `SIGALRM` scatta alla scadenza di un timer misurato sul tempo reale.
- **Controllo Sessione:** `SIGSTOP` “congela” l'esecuzione del processo (inintercettabile e imbloccabile). `SIGCONT` fa ripartire un processo precedentemente fermato e non può mai essere bloccato.
- **Operazioni Errate:** `SIGPIPE` viene generato dal kernel se si tenta di scrivere su una pipe o socket in cui il processo in lettura è terminato o inesistente.
- **Personalizzati:** `SIGUSR1` e `SIGUSR2` sono lasciati alla libera gestione dello sviluppatore per comunicazioni elementari o trigger custom tra processi.

17.3 Il Gestore del Segnale e l'Attesa

Il signal handler è sempre una funzione void che accetta come unico parametro un intero, il quale indica il numero del segnale scatenante (utile se una stessa funzione gestisce segnali diversi).

- **Installazione (signal):** La funzione `signal(signum, handler)` associa la funzione al segnale desiderato. Al posto della funzione, l'handler può essere impostato a `SIG_IGN` per ignorare del tutto l'evento, o a `SIG_DFL` per ripristinare il comportamento predefinito del sistema. Durante l'esecuzione di un handler, il segnale che l'ha scatenato viene bloccato automaticamente dal kernel finché la funzione non termina, per evitare fastidiose interruzioni ricorsive.

- **Pausa (pause e sleep):** Sostituiscono i cicli di attesa attiva che sprecano inutilmente CPU. `pause()` addormenta il processo a tempo indeterminato finché non riceve un qualsiasi segnale. `sleep(seconds)` lo addormenta per un tempo prefissato, ma viene anch'essa interrotta e ritorna prematuramente se un segnale la "sveglia". È pericoloso mescolare `sleep` e `alarm` nello stesso codice, poiché la prima è spesso costruita basandosi proprio sul segnale `SIGALRM` della seconda, causando esiti indefiniti.

17.4 Invio dei Segnali

Un programma può scatenare segnali a piacimento usando le system call fornite.

- **`kill(pid, sig)`:** Serve a inviare un segnale `sig` a un target. L'argomento `pid` è estremamente flessibile: se `>0` colpisce un singolo processo specifico; se `0` colpisce l'intero process group del mittente; se `< -1` invia il segnale ai processi appartenenti al gruppo il cui ID corrisponde al valore assoluto inserito. Se si passa `0` come segnale (segnale nullo), non viene inviato nulla ma il sistema controlla e restituisce eventuali errori di permessi (`EPERM`) o inesistenza del target (`ESRCH`).
- **`alarm(seconds)`:** Predisporre un timer hardware. Alla scadenza dei secondi prefissati, il kernel invierà `SIGALRM` al processo chiamante. Chiamare `alarm(0)` disattiva eventuali allarmi pendenti.

17.5 Impostazione e Blocco (Maschera dei Segnali)

Per superare i gravi limiti della semantica inaffidabile, lo standard POSIX.1 ha introdotto le maschere dei segnali tramite il tipo di dato `sigset_t`. Questo permette di creare sezioni critiche di codice in cui il processo è temporaneamente blindato e immune a determinate interruzioni.

- **Gestione degli Insiemi:** Le funzioni `sigemptyset()` e `sigfillset()` inizializzano una variabile set svuotandola o riempiendola con tutti i segnali del sistema. Le varianti `sigaddset()`, `sigdelset()` e `sigismember()` permettono rispettivamente di includere, escludere o testare la presenza di un singolo segnale nell'insieme.
- **Applicazione del Blocco (`sigprocmask`):** Questa system call interviene sulla maschera attiva del processo, indicando al kernel quali segnali bloccare. L'istruzione agisce in tre modi (parametro `how`): `SIG_BLOCK` somma i nuovi segnali bloccati alla maschera attuale; `SIG_UNBLOCK` li sblocca rimuovendoli; `SIG_SETMASK` sovrascrive totalmente la maschera preesistente con il nuovo array. I segnali bloccati non vanno persi, ma restano congelati in stato "pendente" finché non verranno sbloccati esplicitamente.

18 Intercomunicazione fra processi

Il Capitolo 9 ("Intercomunicazione fra processi") esplora gli strumenti forniti da Unix System V (SysV-IPC) per far dialogare tra loro processi distinti. A differenza di meccanismi come le pipe, l'IPC di System V si basa su oggetti permanenti che risiedono direttamente nel kernel.

Questa architettura porta con sé due difetti storici fondamentali:

- **Permanenza in memoria:** Gli oggetti allocati non vengono distrutti automaticamente quando i processi terminano (come avviene per i file descriptor). Se non vengono cancellati esplicitamente, restano a occupare memoria fino al riavvio del computer.
- **Mancanza di Reference Counting:** Il sistema non tiene traccia di quanti processi stiano usando un oggetto, permettendo l'eliminazione accidentale di una risorsa mentre qualcuno ci sta ancora lavorando.

18.1 Chiavi d'accesso e Comandi di Sistema

Per accedere agli oggetti SysV-IPC (semafori, memoria condivisa o code), il kernel non usa semplici nomi di file, ma identificatori numerici generati dinamicamente e associati a una Chiave (`key_t`).

Per concordare una chiave comune tra processi non imparentati, si usa la funzione `ftok(pathname, proj_id)`. Purtroppo non è un metodo infallibile: incrociando l'inode del file e un ID di progetto, può generare “collisioni” (chiavi duplicate) su sistemi con configurazioni disco particolari.

Se invece i processi sono padre e figlio, si può usare la costante sicura `IPC_PRIVATE` per delegare al kernel la generazione automatica dell'ID.

Da terminale, è possibile monitorare questi oggetti “fantasma” rimasti in memoria usando il comando `ipcs`, e distruggerli a mano usando il comando `ipcrm` seguito dall'ID o dalla chiave dell'oggetto.

18.2 I Semafori System V

A differenza della teoria classica che tratta singoli semafori, System V alloca sempre Insiemi (Array) di Semafori.

- **Creazione (`semget`):** Restituisce l'ID dell'insieme. Il difetto critico è che non è atomica: la funzione crea l'array ma non inizializza i semafori al suo interno, lasciando un buco temporale pericoloso in cui un altro processo potrebbe tentare di usarli.
- **Controllo e Inizializzazione (`semctl`):** Svolge funzioni di configurazione. Richiede un parametro molto particolare: una `union semun` che non è definita di default nelle librerie C e deve essere scritta manualmente dal programmatore nel codice sorgente. Tramite comandi come `SETVAL` (per un singolo semaforo) o `SETALL` (per l'intero array), si imposta il valore iniziale (es. 1 o 0). Si usa anche `IPC_RMID` per cancellare l'insieme.
- **Operatività (`semop`):** Esegue materialmente l'acquisizione o il rilascio. Riceve in pasto un array di strutture `struct sembuf` che indicano l'indice del semaforo e l'operazione (`sem_op`):
 - `sem_op > 0`: Signal (Aggiunge risorse e non blocca mai).

- `sem_op < 0`: Wait (Se il valore è sufficiente lo sottrae, altrimenti il processo si addormenta in attesa).
- `sem_op == 0`: Attesa dello zero (Il processo dorme finché il semaforo non si svuota del tutto).
- **Flag SEM_UNDO**: Una genialata del kernel per arginare i crash. Se il processo muore improvvisamente, il sistema annulla in automatico tutte le operazioni fatte da quel processo sul semaforo, evitando di bloccare gli altri.

18.3 La Memoria Condivisa (Shared Memory)

È la forma di IPC in assoluto più veloce, perché i due processi vedono la stessa identica area di RAM, aggirando del tutto i rallentamenti dovuti ai passaggi intermedi attraverso il kernel (non si usa nessuna funzione di lettura/scrittura per scambiarsi i dati). Essendo memoria pura, è d'obbligo affiancarle dei semafori per evitare race condition.

- **Creazione (`shmget`)**: Alloca nel kernel un segmento di grandezza `size` (arrotondata per eccesso ai limiti delle pagine fisiche).
- **Aggancio (`shmat`)**: Mappa fisicamente la memoria del kernel nello spazio virtuale del processo. È prassi passare `NULL` come indirizzo target, lasciando che sia il sistema operativo a incastrare l'area nel primo spazio vuoto sicuro. Restituisce un puntatore nudo ai dati. Se si usa il flag `SHM_RDONLY`, si mappa in sola lettura.
- **Sgancio e Rimozione (`shmdt` e `shmctl`)**: Terminati i lavori, il processo si scollega con `shmdt`. La distruzione definitiva si ordina con `shmctl(..., IPC_RMID, ...)`: il kernel contrassegna il segmento, ma lo cancella fisicamente solo quando il contatore dei processi collegati scende a zero.

18.4 Code di Messaggi (Message Queues)

Superano i limiti della memoria condivisa grezza, offrendo una lista concatenata gestita dal kernel.

- **Creazione (`msgget`)**: Istanza la coda creando in memoria una struttura `msqid_ds` per tenere traccia dei tempi, dei PID degli ultimi processi mittenti/riceventi e del limite massimo di byte in coda.
- **Struttura Dati (`struct msgbuf`)**: Per viaggiare in coda, il programmatore deve incapsulare i dati in una `struct` il cui primo elemento deve obbligatoriamente essere un intero `long mtype` maggiore di zero (serve a categorizzare il messaggio). Il resto del pacchetto può contenere qualsiasi cosa (stringhe, interi, strutture).
- **Invio (`msgsnd`)**: Inserisce il pacchetto nella lista. Se la coda è piena, il processo si blocca in attesa di spazio, a meno che non si specifichi il flag `IPC_NOWAIT`, che fa fallire subito la chiamata restituendo l'errore `EAGAIN`.
- **Ricezione (`msgrcv`)**: Estrae i dati. La documentazione fornita per questo capitolo si interrompe all'istruzione di lettura `msgrcv`, che normalmente decodifica il messaggio in base alla grandezza `msgsz` e al tipo di filtraggio richiesto `msgtyp`.